Design Projects

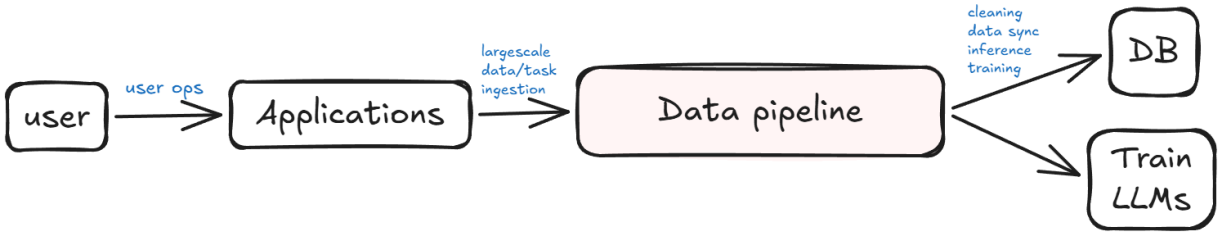
I. Autonomous Distributed Task Processing Engine

MVP: Financial & SEC Intelligence Pipeline

Distributed data pipelines are used to collect, clean, transform, and move massive volumes of information across multiple computers or servers simultaneously.

Core Uses:

- Real-Time Analytics: Processing live streams of user activity, stock prices, or IoT sensor logs instantly as the data is created.
- Machine Learning Operations: Feeding continuous data streams into AI models for distributed training and real-time inference.
- Large-Scale ETL (Extract, Transform, Load): Cleaning and reformatting massive datasets across clusters before loading them into a central data lake or data warehouse.
- Cross-Region Data Syncing: Moving and updating information safely between different company databases and cloud providers around the world.




```
In asynchronous distributed pipelines, worker tasks fail when inputs or
tool parameters suffer from schema drift, malformed payloads, or bad data
types. Standard systems retry endlessly or dump them in a Dead-Letter Queue
(DLQ) for manual triage. We need an autonomous engine that isolates these
failures, diagnoses root causes via an LLM, patches the task arguments
safely, and reconciles state without data loss or race conditions.
```

1. Problem Statement

Modern asynchronous data processing and multi-agent pipelines suffer from severe operational fragility when handling semi-structured inputs, dynamic tool execution, and non-deterministic LLM responses:

- Fragile Tool Execution & Schema Drift: Downstream APIs and tools frequently change return formats or encounter transient parameter errors, causing worker tasks to fail abruptly.
- Poison Pill Blocking: In traditional distributed message queues, unparseable or failing messages repeatedly crash consumer workers or get dumped into a Dead-Letter Queue (DLQ) where they sit indefinitely until manual human intervention.
- Lack of Autonomous Fault Tolerance: Standard retry mechanisms (e.g., exponential backoff) only solve transient network glitches; they cannot resolve payload-level schema mismatches, missing required arguments, or tool contract violations.
- Uncoordinated State & Race Conditions: Running multiple workers and recovery agents without distributed locks and strict idempotency leads to duplicate task execution, state corruption, and split-brain failures.

2. Solution Overview

Build a resilient, distributed task processing engine that combines standard distributed systems primitives (Redis Streams consumer groups, atomic state hashes, idempotency keys, distributed locks) with an autonomous LLM-powered Dead-Letter Queue Recovery Agent:

- Tasks are ingested, partitioned, and consumed asynchronously by worker agents.
- If a worker encounters persistent tool/schema failures after exceeding a retry threshold, the task is safely diverted to a dead letter queue (stream:tasks:dlq).
- A Recovery Agent consumes DLQ events, inspects the error trace and original input against known tool schemas, diagnoses the root cause, and autonomously rewrites/normalizes the task payload.
- Corrected tasks are re-injected into the main pipeline with updated retry budgets, while unrecoverable tasks (e.g., auth failure, missing resource) are marked DEAD with full audit logs.

3. HLD

3.1. Functional Requirements

The system should be able to perform:

- Job ingestion & partitioning: The system must accept batch jobs via an API and partition them into discrete, addressable subtasks.
- Asynchronous execution: Independent worker nodes must consume and execute subtasks using specific tool schemas; The API gateway must accept job requests

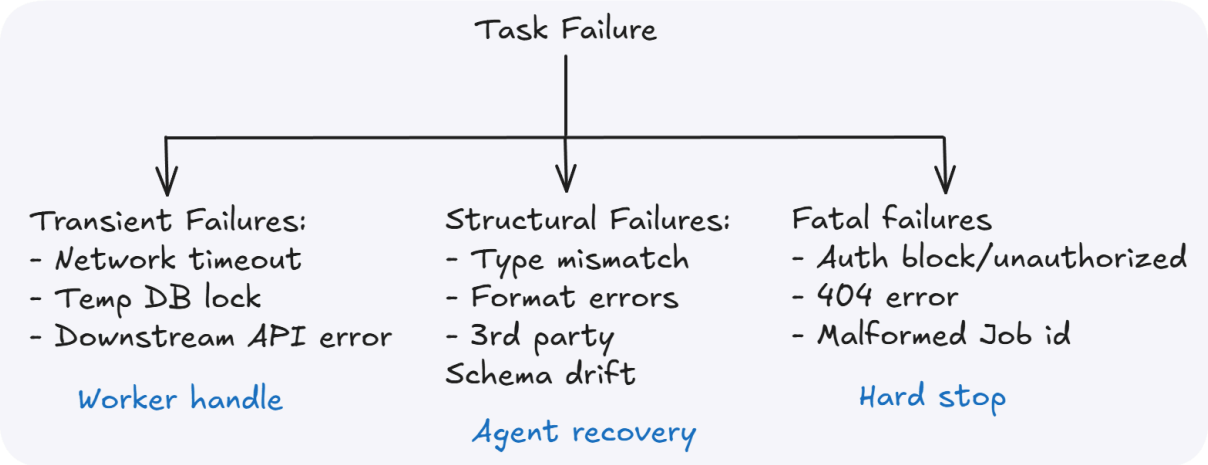
asynchronously, immediately returning an acknowledgment and tracking ID without waiting for downstream task completion.
- Fault isolation: When a task exhausts retries due to non-transient errors (e.g., validation/schema failures), it must be routed to a Dead-Letter Queue with full error context.
- Autonomous remediation: A Recovery Agent must inspect the DLQ, analyze the schema mismatch/traceback using an LLM, and attempt to synthesize a valid, corrected payload.
- Re-injection and Terminal routing: Validated repairs must be re-queued for execution; unrecoverable or repeatedly failing tasks must be routed to a terminal state.
- Central state queueing: The system must expose the real-time state and execution history of any job/task.

3.2. Non-functional

- Scalable
- Reliable and fault-tolerant (At-Least-Once Delivery): No task should be lost if a worker process crashes mid-execution.
- Availability
- Idempotency: Re-executing a task with the same payload/idempotency key must not produce duplicate side effects or duplicate completed records.
- Concurrency Control & Safety: Only one worker/agent can mutate a specific task's state at any given moment; The worker pool must support concurrent task processing across *N* worker instances without cross-task state corruption.
- Bounded Autonomous Execution: Strict caps on recovery attempts to prevent infinite LLM repair loops or runaway token usage.
- Deterministic Validation: The LLM's suggested fix must never be executed blindly; it must be deterministically validated against the target tool's schema before re-injection.

3.3. Failure Scope: What Failures Can We Fall Into?

To decide what the Recovery Agent should fix vs. reject, we group distributed pipeline failures into three clear buckets:



3.4. Constraints

- Worker Retry Cap: Max 3 retries with exponential backoff for Bucket 1 (transient) before routing to DLQ.
- Recovery Attempt Cap: Max 2 LLM remediation attempts per task in the DLQ. If attempt #2 still fails schema validation, force state to DEAD.
- Distributed Lock TTL: Worker lock auto-expires after 10 seconds if a node hard-crashes without releasing its lock.
- PEL Auto-Claim Threshold: Unacknowledged tasks idle for 15 seconds are automatically reassigned to healthy workers.

3.4. Assumptions

3.5. E2E Flow

Phase 1: Ingestion (The Front Door)

1. **Request:** A client submits a payload to the API: `{"job": "financial_analysis", "ticker": "AAPL"}`.
2. **Decomposition:** The Ingestion Engine breaks this into subtasks. For example, Task A: `fetch_metrics`.

3. **State Initialization:** The system writes the initial state to a Redis Hash (`task:state:AAPL_01`: `status=PENDING`, `retry_count=0`).
4. **Enqueue:** The task payload is published to `stream:tasks:pending`.

Phase 2: Worker Execution (The Happy Path)

1. **Claiming Work:** A Worker in the consumer group pulls the task using `XREADGROUP`.
2. **Locking:** The Worker acquires a Redis lock (TTL: 10s) on the task ID to ensure no other worker touches it.
3. **Validation & Execution:**
    - The Worker looks up `fetch_metrics` in the Tool Registry.
    - It passes the payload into the Pydantic schema. It validates perfectly.
    - The tool executes, fetching mock SEC data.
4. **Resolution:** The Worker publishes the result to `stream:tasks:completed`, acknowledges (`XACK`) the pending stream to remove it from the queue, and releases the lock.

Phase 3: The Chaos Path (Schema Drift & DLQ)

1. **Bad Data Ingestion:** A new task arrives, but the payload has a drift error: `{"ticker_symbol": "MSFT", "year": "2024"}` instead of the required `{"ticker": "MSFT", "fiscal_year": 2024}`.
2. **Worker Crash:** The Worker pulls the task. The Pydantic schema throws a `ValidationError` (missing required keys, type mismatches).
3. **DLQ Routing:** Because this is a schema error (not a transient network timeout), the Worker skips standard retries. It packages the original payload, the Pydantic schema requirements, and the stack trace, pushes them to `stream:tasks:dlq`, and `XACK`s the pending queue.

Phase 4: Autonomous Recovery (The Self-Healing Loop)

1. **Discovery:** The Recovery Agent, listening to `stream:tasks:dlq`, picks up the failed MSFT task.
2. **LLM Diagnostics:** It constructs a strict prompt containing:
    - **Target:** `{"ticker": "str", "fiscal_year": "int"}`
    - **Broken Input:** `{"ticker_symbol": "MSFT", "year": "2024"}`
    - **Error:** `KeyError: ticker missing.`

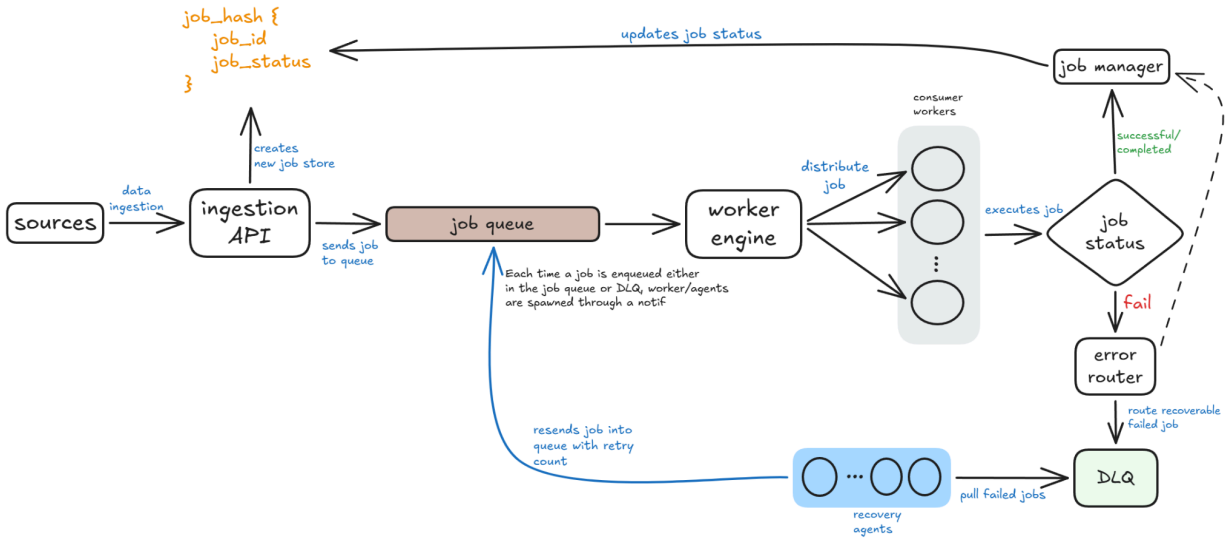
3. **Synthesis:** The local Ollama model outputs a corrected JSON string: `{"ticker": "MSFT", "fiscal_year": 2024}`.
4. **Deterministic Gate:** The Recovery Agent takes the LLM's output and attempts to instantiate the Pydantic model itself.
5. **Re-injection:** Validation passes. The agent increments the `recovery_attempt` count in the Redis Hash, changes the state to `RECOVERING`, and pushes the patched payload to `stream:tasks:retry` (which the main workers treat exactly like the pending queue).

Phase 5: Terminal Failure (The Poison Pill)

If the original payload was completely useless (e.g., `{"company": "Unknown"}`), the LLM might hallucinate or fail to create a valid payload.

4. If the LLM's output fails the Pydantic check, the Recovery Agent rejects it.
5. Once `recovery_attempt` hits 2, the agent gives up, pushes the task to `stream:tasks:dead`, and triggers an alert.

Arch Diagram



6. LLD

```
task-mesh-dlq/
├── docker-compose.yml          # Redis 7, Ollama
├── engine/                     # REUSABLE DOMAIN-AGNOSTIC CORE
│   ├── queue/                  # Redis Streams client, PEL auto-claim
│   ├── concurrency/            # Distributed lock (Redlock implementation)
│   ├── registry/               # Tool registry and schema extractor
│   ├── worker/                 # Generic worker task loop
│   └── recovery/               # Generic DLQ agent & LLM prompt chain
├── plugins/                    # PLUGGABLE DOMAINS
│   └── financial/              # MVP Domain
│       ├── models.py           # Pydantic schemas (DebtRatioArgs, etc.)
│       └── tools.py            # Concrete tool implementations
├── chaos/                      # Test suite for injecting broken payloads
└── tests/                      # Unit and integration tests
```

Data Schemas

Schema Name	Purpose	Key Fields	Data Flow
TaskMessage	Represents a single executable unit of work.	job_id, task_id, tool_name, payload, retry_count, idempotency_key	**From:** Ingestion API or Recovery Agent. **To:** stream:tasks:pending.
DLQMessage	Wraps a failed task with forensic data for the LLM.	original_task, error_message, stack_trace, target_schema, recovery_attempts	**From:** Stream Worker. **To:** stream:tasks:dlq.
TaskState	Central source of truth for a task's lifecycle, stored in Redis.	status (PENDING, IN_PROGRESS, FAILED, DEAD), worker_id, lock_expires_at	**From:** Any active component mutating state. **To:** Redis Hash task:state:{id}.

DomainArgs	Strict contracts for specific tools (e.g., Financial metrics).	Domain-specific fields (e.g., `ticker`, `fiscal_year`).	**From:** `TaskMessage.payload`. **To:** Tool Registry validation.

Core Engine Components

Component	Purpose	Key Methods	Data Flow
IngestionAPI	Receives client requests, generates IDs, and initializes tasks.	`submit_job()`, `get_job_status()`	**Consumes:** HTTP Client Requests. **Produces:** `TaskMessage` to pending stream; `TaskState` to Redis Hash.
ToolRegistry	Central hub linking tool names to their logic and Pydantic schemas.	`register()`, `get_schema()`, `execute_tool()`	**Consumes:** Internal worker queries. **Produces:** JSON Schemas to Recovery Agent; Executed outputs to Workers.
StreamWorker	The primary workhorse that polls queues, locks tasks, and runs tools.	`poll_pending()`, `acquire_lock()`, `handle_success()`, `route_to_dlq()`	**Consumes:** `stream:tasks:pending`. **Produces:** Results to `stream:tasks:completed`; Errors to `stream:tasks:dlq`.
RecoveryAgent	The self-healing loop that diagnoses errors and rewrites payloads.	`poll_dlq()`, `prompt_llm()`, `validate_fix()`, `reinject_task()`, `mark_dead()`	**Consumes:** `stream:tasks:dlq`. **Produces:** Fixed `TaskMessage` to `stream:tasks:retry`; Fatal errors to `stream:tasks:dead`.

ErrorClassifier	Evaluates a failed task's exception to determine its routing destination.	`classify(exception) -> DestinationType`	**Consumes:** Python `Exception` from Worker. **Produces:** Routing decision (`RETRY_LOCAL`, `ROUTE_TO_DLQ`, `ROUTE_TO_DEAD`).

Redis Consumer Group Topology

Stream Name	Assigned Consumer Group	Who is Reading?	What do they do with it?
`stream:tasks:pending`	`cg_main_workers`	All StreamWorkers	Pulls fresh tasks. Uses `XREADGROUP`. Once finished (or routed to DLQ/Dead), the worker calls `XACK` to permanently remove it from the stream.
`stream:tasks:retry`	`cg_main_workers`	All StreamWorkers	Workers treat this exactly like the pending stream. It holds tasks successfully patched by the Recovery Agent.
`stream:tasks:dlq`	`cg_recovery_agents`	All RecoveryAgents	Pulls tasks specifically flagged for LLM schema repair.
`stream:tasks:completed`	*(No consumer group)*	Reporting / UI Services	Acts as an append-only audit log of successful executions. Services can tail this stream to update user dashboards.

stream:tasks:dead	*(No consumer group)*	Alerting Services	Acts as an append-only log of fatal failures. Can trigger PagerDuty or email alerts.

State Transitions & Triggers

Every time a component updates the state in Redis, it executes an `HSET task:state:{id} status {NEW_STATE}` command.

From State	To State	Trigger / Condition	Component Responsible
(None)	*PENDING*	Client API submits a new job.	Ingestion API
PENDING	*IN_PROGRESS*	Worker pulls from stream:tasks:pending and successfully acquires the distributed lock.	Stream Worker
IN_PROGRESS	*COMPLETED*	Tool executes successfully, and output is validated.	Stream Worker
IN_PROGRESS	*PENDING*	Tool fails, but Error Classifier says RETRY_LOCAL and retry_count < max. (Lock is released).	Stream Worker
IN_PROGRESS	*IN_DLQ*	Tool fails, Error Classifier says ROUTE_TO_DLQ (e.g., Pydantic schema mismatch).	Stream Worker

IN_PROGRESS	*DEAD*	Tool fails, Error Classifier says `ROUTE_TO_DEAD` (e.g., Invalid Auth, 404).	Stream Worker
IN_DLQ	*RECOVERING*	Recovery Agent pulls from `stream:tasks:dlq` and acquires the lock.	Recovery Agent
RECOVERING	*PENDING*	LLM successfully generates a valid JSON payload. Task is pushed to `stream:tasks:retry`.	Recovery Agent
RECOVERING	*DEAD*	LLM outputs invalid JSON *or* `recovery_attempts` hits the maximum limit.	Recovery Agent

The Lifecycle Commands

To make this topology resilient, workers execute three specific Redis mechanisms:

- **`XREADGROUP`** (The Pull): A worker asks its group, *"Give me 1 task that no one else is currently working on."* Redis gives the task and puts it in the *Pending Entries List (PEL)*.
- **`XACK`** (The Acknowledge): Once the worker finishes (either success, DLQ routing, or Dead routing), it sends an `XACK`. This deletes the task from the PEL.
- **`XAUTOCLAIM`** (The Sweeper): If a worker pulls a task but gets killed (OOM crash, unplugged server) before sending an `XACK`, the task sits in the PEL forever. A background thread periodically runs `XAUTOCLAIM` to say, *"Give me any tasks in the PEL that haven't been ACKed in 15 seconds,"* reassigning them to healthy workers.

7. Implementation

We will build this iteratively, validating each component before moving to the next.

Phase 1: Infrastructure & Environment Setup

- **Goal:** Establish the local runtime and dependencies.
- **Tasks:**

    - Write docker-compose.yml for Redis and Ollama.
    - Set up the Python virtual environment and requirements.txt (FastAPI, Redis-py, Pydantic, HTTPX).
    - Verify the local LLM model pulls successfully.

Phase 2: Core Distributed State & Queue Engine

- **Goal:** Build the domain-agnostic message broker layer.
- **Tasks:**
    - Implement the Redis connection pool.
    - Create the Stream Publisher (to enqueue tasks).
    - Build the Redis Hash State Manager (to track PENDING → IN_PROGRESS, etc.).
    - Implement the distributed locking mechanism (Redlock pattern).

Phase 3: The Worker Pool & Financial Plugin

- **Goal:** Execute tasks and handle failures deterministically.
- **Tasks:**
    - Define the Pydantic schemas for the Financial MVP (FetchMetricsArgs, etc.).
    - Implement the ToolRegistry to map task types to functions.
    - Build the StreamWorker loop with the Blocking Pull (XREADGROUP).
    - Implement the ErrorClassifier to route transient errors to retries and schema errors to the DLQ.

Phase 4: The Autonomous Recovery Agent

- **Goal:** Implement the self-healing LLM loop.
- **Tasks:**
    - Build the RecoveryAgent loop to consume from stream:tasks:dlq.
    - Write the dynamic prompt constructor (injecting the Pydantic JSON schema and error trace).
    - Integrate the local Ollama API to generate the patched payload.
    - Add the deterministic validation gate before re-injecting to stream:tasks:retry.

Phase 5: API Gateway & Chaos Testing

- **Goal:** Expose the system and prove it heals.
- **Tasks:**
    - Build the FastAPI endpoints (POST /jobs, GET /jobs/{id}).
    - Write a chaos script that intentionally submits malformed JSON payloads.
    - Verify end-to-end flow: Submission → Crash → DLQ → Repair → Success.

Project Folder Structure

This layout keeps the execution engine completely agnostic to the financial logic.

```
task-mesh/
├── docker-compose.yml        # Defines Redis and Ollama services
├── requirements.txt          # Python dependencies
├── .env                      # Environment variables (Redis URL, Ollama Host)
│
├── api/                      # PHASE 5: FASTAPI GATEWAY
│   ├── __init__.py
│   └── routes.py             # Endpoints for job submission and status polling
│
├── engine/                   # PHASE 2 & 4: DOMAIN-AGNOSTIC CORE
│   ├── __init__.py
│   ├── redis_client.py       # Redis connection pool and setup
│   ├── state.py              # Redis Hash CRUD operations (TaskState)
│   ├── queue.py              # Stream publishing, XREADGROUP, XACK logic
│   ├── locks.py              # Distributed lock implementation
│   ├── worker.py             # The Main Worker consumer loop
│   ├── classifier.py         # Error Classifier (Transient vs DLQ vs Fatal)
│   └── recovery.py           # The DLQ Recovery Agent & Ollama integration
│
├── plugins/                  # PHASE 3: BUSINESS LOGIC
│   ├── __init__.py
│   ├── registry.py           # Dynamic tool registry for Pydantic mapping
│   └── financial/            # The specific MVP domain
│       ├── __init__.py
│       ├── models.py         # Pydantic schemas (FetchMetricsArgs, etc.)
│       └── tools.py          # Actual python functions (fetching mock data)
│
└── tests/
    ├── test_worker.py
    └── chaos_injector.py     # Script to spam the API with broken payloads
```

task-mesh/
├── docker-compose.yml   # Defines Redis and Ollama services
├── requirements.txt   # Python dependencies
├── .env   # Environment variables (Redis URL, Ollama Host)
│
├── api/   # PHASE 5: FASTAPI GATEWAY
│   ├── __init__.py
│   └── routes.py   # Endpoints for job submission and status polling
│
├── engine/   # PHASE 2 & 4: DOMAIN-AGNOSTIC CORE
│   ├── __init__.py
│   ├── redis_client.py   # Redis connection pool and setup
│   ├── state.py   # Redis Hash CRUD operations (TaskState)
│   ├── queue.py   # Stream publishing, XREADGROUP, XACK logic

```
│   ├── locks.py            # Distributed lock implementation
│   ├── worker.py           # The Main Worker consumer loop
│   ├── classifier.py       # Error Classifier (Transient vs DLQ vs Fatal)
│   └── recovery.py         # The DLQ Recovery Agent & Ollama integration
│
├── plugins/                # PHASE 3: BUSINESS LOGIC
│   ├── __init__.py
│   ├── registry.py         # Dynamic tool registry for Pydantic mapping
│   └── financial/          # The specific MVP domain
│       ├── __init__.py
│       ├── models.py       # Pydantic schemas (FetchMetricsArgs, etc.)
│       └── tools.py        # Actual python functions (fetching mock data)
│
└── tests/
    ├── test_worker.py
    └── chaos_injector.py   # Script to spam the API with broken payloads
```

Technologies & Architecture Choices

Here is the finalized stack for the Distributed Multi-Agent Task Mesh MVP and why each tool was selected:

Python (3.11+)

- Reason: It is the industry standard for AI and agentic workflows. Its native support for asynchronous I/O (`asyncio`) makes it perfect for building non-blocking worker loops that poll queues without locking up system resources.

FastAPI

- Reason: It is a high-performance web framework that natively uses `async` and seamlessly integrates with Pydantic. It automatically generates a Swagger UI, which will act as our makeshift dashboard for submitting chaos tests.

Redis (Version 7+)

- Reason: It acts as a "triple threat" for this architecture. We use **Redis Streams** for the message broker (replacing Kafka/RabbitMQ), **Redis Hashes** for the central state machine (replacing PostgreSQL), and **Lua scripts** for distributed locking (Redlock). It keeps the infrastructure footprint incredibly small.

Ollama (Local LLM Engine)

- Reason: It allows us to run models like `llama3.1:8b` or `qwen2.5:7b` entirely locally via Docker. This means zero API costs, zero rate limits during testing, and no latency bottlenecks when the Recovery Agent loops.

Pydantic (v2)

- Reason: It is the backbone of our schema contracts. It provides strict runtime validation to catch broken payloads instantly, and it natively exports `.model_json_schema()`, which we need to feed to the LLM so it knows how to fix the broken data.

Docker & Docker Compose

- Reason: It guarantees a reproducible local environment. A single `docker-compose up` command will spin up the message broker (Redis) and the LLM engine (Ollama) with the correct network bindings, completely isolated from your host OS.